



Project title: Banking System

Course title: Programming in Java

Code: CSE2006

Course type: Flipped course Course

Credits: 3

Professor: MANORMA CHOUHAN

Slot: F11+F12

Submitted by : S G SUDHARSHAN

Registration number: 25BAI10706

date: 18-09-2026

INTRODUCTION:

A Bank System is a structured framework that manages financial transactions, customer records, and various banking operations. It allows users to perform essential activities such as depositing money, withdrawing funds, checking account balances, and transferring amounts securely. Modern banking systems are designed to ensure accuracy, speed, and safety in handling customer data and financial activities.

In today's digital world, bank systems rely heavily on technology to provide seamless and efficient services. Automated software solutions help reduce manual errors, improve customer experience, and support online banking features. These systems also ensure data security, maintain transaction history, and enable banks to manage millions of customer accounts.

PROBLEM STATEMENT:

Managing banking operations manually often leads to errors, delays, and difficulty in maintaining accurate financial records.

Traditional methods of handling deposits, withdrawals, account creation, and balance management are time-consuming and inefficient, especially when dealing with many customers. There is also a high risk of data mismanagement, unauthorized access, and lack of transparency in transaction tracking.

To overcome these challenges, there is a need for a computerized Banking System that automates essential banking functions. The system should allow users to create accounts, deposit and withdraw money, view transaction history, and check balances securely and efficiently. It must ensure data accuracy, improve service speed, and maintain high security standards while providing a simple and user-friendly interface for both customers and administrators.

PROJECT OBJECTIVES :

- To automate basic banking operations such as account creation, deposits, withdrawals, balance inquiry, and transaction recording in an efficient and error-free manner.
- To provide a secure system that protects customer data, prevents unauthorized access, and ensures safe handling of transactions.

- To improve accuracy and reduce manual errors by maintaining digital records of all accounts and financial activities.
- To enhance user experience with a simple, intuitive, and user-friendly interface for both customers and administrators.
- To ensure fast processing of transactions so that banking operations can be performed quickly and reliably.
- To maintain detailed transaction history and provide transparency for both users and bank management.
- To support scalability, allowing the system to manage multiple customer accounts and increasing data volume without performance issues.
- To reduce workload on staff by automating repetitive banking tasks and improving overall operational efficiency.

FUNCTIONAL REQUIREMENTS:

1. User-System Interaction:

- Users enter a request such as creating an account, depositing money, withdrawing money, or checking
- System verifies and processes the request by checking account details, validating inputs, and performing the required transaction.
- System displays the result by showing confirmation messages, updated balance, or error notifications.

2. Data Input:

- User Information Input: The user provides personal details such as name, PIN, and Minimum balance during account creation.
- Account Details Input: The system collects account-related data like account name initial deposit amount
- Transaction Input: Users enter transaction-related data such as deposit amount, withdrawal amount, or account name for balance inquiry.

3. Data processing:

- Deposit Process The system receives the user's account number, Date and deposit amount. It verifies if the account exists and checks for valid input. The system adds the deposit amount to the existing balance and updates the account records.
- Withdraw Process The user enters the account name, Date and withdrawal amount. The system verifies account validity and checks whether sufficient balance is available.
- Account Balance Process The user inputs the account number to view the balance. The system validates the account and retrieves the stored balance information. It displays the current balance to the user.

- **Account List Process** The system gathers all stored customer account records. It organizes and retrieves account details such as account number, name, and balance.

4. Output:

- The System displays the Balance of the account.
- The System displays the account list,

5. Data storage:

- The input data entered by user is stored by the system for the future reference and future account balance.

NON-FUNCTIONAL REQUIREMENTS :

1. Performance:

- The system shall provide the details of particular account details within a couple seconds.
- The system shall provide the list of accounts list details.

2. Security:

- The system will not hold / store any sensitive data of the user.

- The system shall be secure against all the common web vulnerabilities.

3. Usability:

- The system is made to be simple and easy for the user to use, and the set of instruction will guide the user throughout Banking process.

4. Reliability:

- The system will provide accurate account details.
- And the system can be accessed anywhere around the world with a python code executer.

5. Maintainability:

- The system is written with a maintainable and understandable code with comment lines.
- The system has proper documentation so that it can be updated in the coming future.

6. Error handling:

- If the system detects any error in the inputs entered by the user, it displays an error message and requests' the user to try entering the inputs again.

7. Resource efficiency:

- The system is designed to minimise power consumption and optimize battery life.
- The system shall require less than 10MB of storage space for optimised running.

- The system doesn't require any network bandwidth so it can be accessed even in places without proper internet connection.

SYSTEM ARCHITECTURE :

- **Presentation Layer:**

Built using **Java Scanner and console interface**, where the user enters details such as account name, PIN, deposit amount, withdrawal amount, and other required inputs.

- **Application Layer:**

The **Java program logic** processes the user inputs, performs operations such as deposit, withdrawal, balance calculation, PIN verification, and validation.

- **Data Layer:**

Uses **Java ArrayList and class objects** to store account details, deposits, withdrawals, and balance information during program execution.

- **Output Renderer:**

The **Java console output** dynamically displays the results, including account details, deposit records, withdrawal records, and current balance.

DESIGN DIAGRAM :

1. Workflow:

- The VIT BPL Bank application follows a simple menu-driven workflow. The program starts by displaying a

welcome message and then presents the main menu to the user. The user can choose to create an account, deposit money, withdraw money, check the balance, or list all accounts. Based on the selected option, the corresponding function is executed and the required information is stored using ArrayList. After completing an operation, the user is returned to the main menu. The process continues until the user selects the Exit option.

2. Data flow Diagram :

- Create Account:
Create a Person bank account details.
- Validation :
Validate the PIN to the Particular Account.
- Computation:
Calculate the Account balance Minimum balance +
Deposit amount - Withdrawal amount.
- Output :
Displays the processed data, the results and then allows the user to calculate account balance

3.Component diagram:

- Input form: Collects Bank account details from the user.
- Validation module : Checks all input ranges and types.
- Calculation core : Compute Account Balance.

- Output display : Presents the account details.

DESIGN DECISIONS & RATIONALE :

1. Programming Language: Java

- **Rationale:** Java is a simple, object-oriented and platform-independent programming language. It is suitable for implementing small console-based projects like the VIT BPL Bank system.

2. User Interface: Command Line Interface (CLI)

- **Rationale:** A simple console-based interface is sufficient for this project. The `Scanner` class is used to take user input and display the results through the command prompt or terminal.

3. Data Storage: In-Memory Data Storage

- **Rationale:** The project does not require permanent data storage. `ArrayList` is used to store account, deposit, and withdrawal details temporarily while the program is running.

4. Calculation Algorithm: Simple Formula-Based Calculation

- **Rationale:** The balance is calculated using a simple formula:

Current Balance = Minimum Deposit + Total Deposits – Total Withdrawals

The program uses loops to calculate the total deposited and withdrawn amounts for each account.

5. Error Handling

- **Rationale:** Basic error handling is used for situations such as an account not being found, an invalid menu option, an invalid PIN length, and insufficient minimum deposit. The program is designed for users to enter valid input.

IMPLEMENTATION DETAILS

1. Language: Java

- **Implementation:** The VIT BPL Bank system is implemented using Java with classes, methods, ArrayList, and Scanner.

2. Development Environment: VS Code / Command Prompt

- **Implementation:** The program is developed and executed using Visual Studio Code and can be run through the Command Prompt or PowerShell.

3. Module: VITBPLBank.java

- **Implementation:** The complete banking application is implemented in a single Java source file named VITBPLBank.java.

4. Functions/Methods: Banking Operations

- **Implementation:** The program uses separate methods for each banking operation:
 - `createAccount()` – creates a new bank account.
 - `deposit()` – records deposited amounts.
 - `withdraw()` – records withdrawn amounts.
 - `balance()` – calculates and displays the current balance.
 - `listAccounts()` – displays all registered bank accounts.
 - `mainMenu()` – provides the main menu and controls the program flow.

5. Implementation Steps:

- Implement the bank account details using `ArrayList`.
- Implement account creation with name, PIN, and minimum deposit.
- Implement deposit and withdrawal operations with date and amount.
- Allow the user to check the account balance.
- Allow the user to view all bank accounts.
- Calculate the balance using minimum deposit, total deposits, and total withdrawals.
- Test the program using sample inputs and verify that the displayed results are correct.

TESTING APPROACH

1. Unit Testing:

- Manually tested individual methods such as `createAccount()`, `deposit()`, `withdraw()`, `balance()`, and `listAccounts()` using known sample inputs to verify that each operation works correctly.

2. System Integration Testing:

- Tested the complete banking system through the main menu by performing different operations such as creating accounts, depositing amounts, withdrawing amounts, checking balances, and listing all accounts.

3. Validation Testing:

- Tested boundary conditions such as a **4-digit PIN**, minimum deposit of **₹500**, deposit/withdrawal amounts, valid account names, and checking the balance of an account that does not exist.

4. User Flow Testing:

- Tested the complete user flow from account creation to deposit, withdrawal, and balance checking. Invalid menu

options and incorrect account names were also tested to ensure appropriate error messages are displayed.

5. Peer Review:

- The program was reviewed by peers to verify the correctness of banking operations, balance calculations, program flow, and overall reliability.

SCREENSHOTS/RESULTS:

```
PS C:\S G SUDHARSHAN DETAILS\vityarthi_java> java VITBPLBank.java
===== WELCOME TO VIT BPL BANK =====

Welcome to vit bpl bank
1) Create account
2) Deposit
3) Withdraw
4) Check balance
5) List all accounts
0) Exit
Select an option: |
```

Welcome to vit bpl bank

- 1) Create account
- 2) Deposit
- 3) Withdraw
- 4) Check balance
- 5) List all accounts
- 0) Exit

Select an option: 1

===== Create Account =====

Full name: S G SUDHARSHAN

Enter a 4digit pin:2007

Enter the min balance:4000

SUCCESSFULLY ADDED

Name	Pin	Min_deposit
S G SUDHARSHAN	2007	4000.0

Welcome to vit bpl bank

- 1) Create account
- 2) Deposit
- 3) Withdraw
- 4) Check balance
- 5) List all accounts
- 0) Exit

Select an option: 2

===== The Deposited Amount =====

Enter Account Name: S G SUDHARSHAN

Enter The Date (YYYY-MM-DD): 2026-09-16

Enter The Deposit Amount: 5000

SUCCESSFULLY DEPOSITED

Name	Dep_Date	Dep_amt
S G SUDHARSHAN	2026-09-16	5000.0

Welcome to vit bpl bank

- 1) Create account
- 2) Deposit
- 3) Withdraw
- 4) Check balance
- 5) List all accounts
- 0) Exit

Select an option: 3

===== The Withdrawn Amount =====

Enter Account Name: S G SUDHARSHAN

Enter The Date (YYYY-MM-DD): 2026-09-16

Enter The Withdraw Amount: 4000

SUCCESSFULLY WITHDRAWN

Name	Wth_Date	Wth_amt
S G SUDHARSHAN	2026-09-16	4000.0

Welcome to vit bpl bank

- 1) Create account
- 2) Deposit
- 3) Withdraw
- 4) Check balance
- 5) List all accounts
- 0) Exit

Select an option: 4

===== Your Balance =====

Enter Account Name: S G SUDHARSHAN

Name	Pin	Min_deposit
S G SUDHARSHAN	2007	4000.0

Total Deposited: 5000.0
Total Withdrawn: 4000.0
Current Balance: 5000.0

```
Welcome to vit bpl bank
1) Create account
2) Deposit
3) Withdraw
4) Check balance
5) List all accounts
0) Exit
Select an option: 5
=====All Bank Accounts=====

Name          Pin          Min_deposit
S G SUDHARSHAN      2007          4000.0
```

```
Welcome to vit bpl bank
1) Create account
2) Deposit
3) Withdraw
4) Check balance
5) List all accounts
0) Exit
Select an option: 0
Goodbye!
```

CHALLENGES FACED

1. Finding an Account:

- Finding a particular bank account and retrieving its details was a challenge during implementation. This was solved by using `ArrayList` and loops to search for the account name and identify the corresponding account details.

2. User Error Handling:

- Handling invalid user inputs such as an incorrect PIN length, insufficient minimum deposit, invalid menu

options, and non-existing account names required additional validation and conditional statements.

3. Dynamic Updates:

- Updating deposit and withdrawal records dynamically for the selected account was a challenge. This was implemented using separate `ArrayList` collections for account, deposit, and withdrawal information.

4. Balance Calculation:

- Calculating the correct current balance based on the account's minimum deposit, total deposits, and total withdrawals required careful implementation of loops and arithmetic operations.

5. Transparency:

- Providing clear step-by-step messages for account creation, deposits, withdrawals, and balance calculations was important so that users could understand the operations rather than only seeing the final result.

LEARNING AND KEY TAKEAWAYS :

1. Implementation:

Learnt how to implement the Logic with adequate knowledge to solve the real-world problems,

2. Automation

Finding the results This would have took a lot of time and energy if it was done manually but now within seconds with the help of this finding balance we can get the results automatically in a few seconds.

3. Testing :

Coding part is easy if we know the logic behind it but we can only know our code is correct when it tests run it with multiple values, so I understood that testing a system is as important as designing a system.

4. Communication:

The system's ability of guidance has been improved, and error messages are directly displayed in the output UI for optimal user support.

FUTURE ENHANCEMENTS

1. Data Persistence:

- Add database or file-based storage such as **MySQL, SQLite, or CSV** so that account, deposit, and withdrawal details are saved permanently and are not lost when the Java program is closed.

2. User Interface:

- Develop a **GUI or web-based interface** using technologies such as Java Swing, JavaFX, or a Java web framework to provide a more attractive and user-friendly banking experience instead of using the Command Line Interface.

3. Analytics:

- Add transaction analysis and graphical dashboards to display information such as **total deposits, withdrawals, current balance, and transaction history** for easier understanding of account activity.

4. Export:

- Add features to export **bank account details, deposit records, withdrawal records, and balance reports** into formats such as **PDF or Excel**, with an option to share reports through email.

5. Security:

- Improve security by adding **PIN authentication, password protection, encrypted user data, and transaction verification** to protect customer account information.

REFERENCE:

1. VIT BHOPAL Academic guidelines for Grading and Credit calculation.
2. VITYARTHI and CSE2006 course for JAVA coding skills.
3. Google for learning about Documentation.
4. Peer and mentor feedbacks from,
 - * FACULTY: MANORMA CHOUHAN,
 - * Gokul - 22BCG10045,
 - * Gopi -23BSA10162,
 - * Gokul Prasad. K - 25BAI10148,
 - * Bavesh. P-25BAI10093,
 - * DILIP S-25BAI10590